Does Agent Decomposition Pay Under a Shared Budget? A Preregistered Response Surface on Function Calling

Daniil Privezentsev[1*]

[1*]Independent Researcher, Russia.

Corresponding author(s). E-mail(s): daniil.privezentsev@gmail.com;

Abstract

Agent-system comparisons often give decomposed systems more inference compute than a unified baseline. We isolate architecture from completion budget in a preregistered response-surface experiment using the official Berkeley Function-Calling Leaderboard checker. Twenty held-out tasks, four OpenRouter endpoints, three orchestration modes, and matched episode budgets $\boldsymbol{T} \in \mathbf{\{512, 1024, 2048\}}$ produce 720 intention-to-treat episodes and 5622 model calls. A router is trained only on a disjoint 40-task routing set and frozen before confirmatory outcomes. We report task-stratified bootstrap intervals, Holm-adjusted tests, an equivalence analysis, token integrity, and provider cost. Overall accuracy is only 0.049 and 85–97.5% of episodes in each mode–budget cell end in an invalid final call, so the result is explicitly floor-dominated. Per budget doubling, the paired decomposition effects are -0.031 for plan–execute–review and -0.050 for a separated team; none favours decomposition. The frozen router, best-static policy, and always-unified policy select the same actions, with regret difference 0.000 $[0.000, 0.000]$. A preregistered 32-task validation stopped at 352/384 unscored episodes at the disclosed hard spending ceiling and is not quality evidence. A separate feasibility arm passes 7/12 Terminal-Bench 2.0 tasks ($[0.320, 0.807]$) with 4 infrastructure failures; it is not a topology comparison. The contribution is a fully disclosed negative/floor result and reproducible matched-budget artifact, not a new orchestration algorithm or a state-of-the-art claim.

Keywords: LLM agents; function calling; matched compute budget; preregistration; adaptive orchestration; held-out routing; Terminal-Bench.

1 Introduction

An "agent architecture" determines whether one model call solves a task end to end or whether planning, execution, and review are split across calls or roles. More roles can expose parallelism and error checking, but they also repeat context, fragment the answer budget, and introduce coordination failures. Published comparisons often change both architecture and inference compute at once. Consequently, a higher score cannot be attributed to decomposition alone.

We study *agent granularity*: how much a task is partitioned into independently prompted stages. For every task–model pair we execute a unified solver (G0), a sequential plan–execute–review pipeline (G1), and a separated team (G3), while enforcing the same total completion-token ceiling T. We sweep $T = 512, 1024, 2048$ instead of selecting one convenient budget. Quality is checked by the pinned official BFCL evaluator, and all persistent generation/evaluation failures remain zero-valued intention-to-treat observations.

Research Question 1. *At a fixed total completion budget, how do G1 and G3 change official function-calling accuracy relative to G0?*

Research Question 2. *Does the paired effect of decomposition change when the episode budget doubles?*

Research Question 3. *Are budget effects stable across four model families and four BFCL task categories?*

Research Question 4. *Does a router frozen on disjoint tasks beat the preregistered best static mode on held-out tasks, and can the planned second-set validation answer the same question?*

The contribution is empirical and methodological rather than a priority claim for adaptive orchestration. First, we provide a preregistered $20 \times 4 \times 3 \times 3$ response surface and disclose an attempted central-budget validation on 32 additional tasks that stopped incomplete and remains unscored. Second, the router is trained on a disjoint 40-task matrix and serialized before confirmatory outcomes exist; its held-out null is reported as a degeneracy result, not as evidence of adaptive gain. Third, every claim is connected to raw calls, official checker output, integrity summaries, generated tables, and figures. Fourth, a descriptive Terminal-Bench 2.0 arm tests one coding agent on 12 pinned repository tasks and records agent, environment, and verifier failures separately. A preceding 40-task synthetic study is retained as Study 0 because it motivated the design, but it is not allowed to strengthen the real-benchmark confirmatory claims.

Claim boundary.

This is a response-surface and replication study, not a new learning algorithm and not a BFCL or Terminal-Bench leaderboard submission. It covers four hosted models, selected BFCL categories, one provider snapshot, and one Codex/Harbor repository

configuration. It can support statements about matched-budget decomposition in this frozen panel; it cannot establish universal agent scaling laws, state-of-the-art performance, or A*-venue acceptance.

2 Related Work

2.1 Single-agent baselines and tool use

ReAct established the practical pattern of interleaving reasoning and acting [2]; in our setting its unified controller is closest to G0. Reflexion and Self-Refine show the value of iterative self-correction without weight updates [3, 4]; review in G1/G3 is an external form of the same idea, paid for from the shared budget.

2.2 Internal roles in one context

Introspection of Thought executes a programmatically specified role dialogue inside a single prompt and reports lower token cost than external reasoning frameworks [18]. Study 0 included this one-context internal-role condition; the real-benchmark phase focuses on externally decomposed calls because those expose ledger and coordination effects.

2.3 Multi-agent and elastic systems

MegaAgent demonstrates dynamic agent spawning with monitoring and limits [9]; G3 is a deliberately simplified coordinator–workers–synthesizer representative under a hard budget. G1 represents fixed sequential external roles with repeated context transmission.

2.4 Adaptive orchestration, budgets, and architecture search

The closest line of work selects system structure per query: DyLAN dynamically chooses a team [5]; GPTSwarm optimizes an agent graph [8]; AgentPrune removes messages [14]; ADAS, AFlow, MASS, and SEMAG search workflows, prompts, or topologies [15, 16, 21, 29]; MaAS samples query-dependent architectures with cost awareness [17]; AdaptOrch and DyTopo route topologies [22, 23]. Still closer budgeted formulations already exist: BAMAS jointly selects models and topology under a cost budget [25], MACA learns a task- and budget-conditioned structural prior [26], and ZEBRA allocates a fixed budget across pipeline phases [27]. Our novelty is therefore neither adaptive architecture selection nor budget awareness.

2.5 Budget-honest comparisons and scaling

Tran and Kiela show that single-agent systems can outperform MAS under equal thinking-token budgets [24]. Kim et al. systematize agent-system scaling with standardized tools and budgets [20]. Conversely, DebateCoder reports gains from adaptively gated multi-agent discussion on a small Pangu-1B model [28]. Our protocol is not a priority claim but an independent test of these conflicting findings: four

hosted endpoints, an identical completion ceiling for every compared structure, open tasks and traces, and publication of negative results.

2.6 Benchmarks and evaluation

HumanEval [1], BigCodeBench [10], SWE-bench [6], SWE-bench Verified [11], and Multi-SWE-bench [19] span the spectrum of software tasks; critiques of SWE-bench point to leakage and weak tests [13]. Our synthetic precursor controls task features but has limited external validity, so the revised evidence uses official BFCL tasks and a pinned Terminal-Bench subset. LLM-as-a-Judge [7, 12] is not used: quality comes from executable benchmark checkers.

The revised study instead centres on BFCL's executable function-call checker [30] and uses Terminal-Bench with Harbor for a repository-level stress test [31, 32]. ToolOrchestra is the closest model/tool-routing analogue [33]; the Harness Effect and the Token Budgets catalogue independently motivate treating orchestration and budget enforcement as measured system components [34, 35]. Unlike leaderboard studies, we estimate paired topology effects under the same episode ledger and make no state-of-the-art claim.

3 Terms, estimands, and hypotheses

Definition 1 (Agent and tool call)**.** *An* agent *is a model-driven controller that receives an observation, chooses one or more function calls, and may continue from the returned tool state. A* tool call *is a structured function name plus JSON arguments; the official evaluator, not a language-model judge, determines whether it is correct.*

Definition 2 (Granularity)**.** Agent granularity *is the degree to which one episode is divided into separately prompted roles. G0 uses one unified controller; G1 uses planning, execution, and review in sequence; G3 uses a coordinator and isolated worker roles before synthesis. These labels identify prompt topologies, not numbers of distinct neural networks.*

An *episode* is one task–model–mode–budget execution. Its registered completion budget T covers all calls:

$$\sum_{r=1}^{R} c_r \le T, \tag{1}$$

where R is the number of model calls and c_r is the provider-reported completion count of call r, including hidden reasoning when reported. Thus a three-call system does not receive three times the allowance. Prompt tokens, latency, and monetary cost are measured but do not relax Eq. (1).

The primary outcome $Y \in \{0, 1\}$ is official BFCL correctness. Persistent transport, decoding, or evaluator failures are kept with $Y = 0$ under *intention to treat* (ITT): every registered episode remains in the denominator. For mode $g \in \{\mathrm{G1}, \mathrm{G3}\}$ and

budget T, the paired effect is

$$\Delta_g(T) = \frac{1}{|\mathcal{T}||\mathcal{M}|} \sum_{\tau \in \mathcal{T}} \sum_{m \in \mathcal{M}} \left[Y_{\tau m g T} - Y_{\tau m \mathrm{G0} T} \right]. \tag{2}$$

In plain language, each task and model is compared with itself; positive Δ favours decomposition. The budget-interaction estimand is the change in this paired effect per one-unit increase of $\log_2 T$, i.e., per budget doubling. H1 predicts a positive interaction, H2 tests each $\Delta_g(T) = 0$, and the two H2 families receive Holm correction. A 90% two-one-sided-test interval wholly inside $[-0.03, 0.03]$ is evidence of practical equivalence, not merely failure to reject a difference.

For routing, a cell's preregistered utility is

$$U = Y - 0.02 \log_2(1 + t/128) - 0.10 I_{\text{invalid}}, \tag{3}$$

where t is total provider-reported tokens and $I_{\text{invalid}} = 1$ for a malformed tool call. The penalty makes accuracy primary while breaking ties in favour of fewer tokens and valid structure. The oracle chooses the largest observed U within a task–model–budget block. Policy regret is $U_{\text{oracle}} - U_{\text{policy}}$; zero is best. H3 is the mean regret of the frozen router minus the mean regret of the preregistered best-static policy, so a negative value favours routing.

Definition 3 (Held-out test, bootstrap, and confidence interval)**.** *A* held-out *task is never used to fit or select the evaluated policy. The* task bootstrap *repeatedly samples whole tasks with replacement and recomputes an estimand; it approximates how that estimand varies across tasks. A 95% bootstrap confidence interval is the central 95% of those resampled estimates. It is not a 95% posterior probability that the fixed parameter lies inside the reported endpoints.*

The *Holm correction* orders a family of p-values and tests them against successively less stringent thresholds, controlling the probability of at least one false rejection in that family. A *router* is a policy that selects G0, G1, or G3 before observing their outcomes; the oracle is only an after-the-fact reference.

4 Preregistered design

4.1 Freeze order and task sets

The repository history exposes the order of commitments rather than asking the reader to trust a retrospective statement. The model panel, prompts, budget ledger, and 40 routing tasks were tagged before routing generation. The serialized router was committed before confirmatory dispatch. A separate 32-task validation manifest was self-hashed and tagged before any validation generation. A provider-limit incident produced no answer and zero cost; its recovery rule was likewise tagged before replacement dispatch.

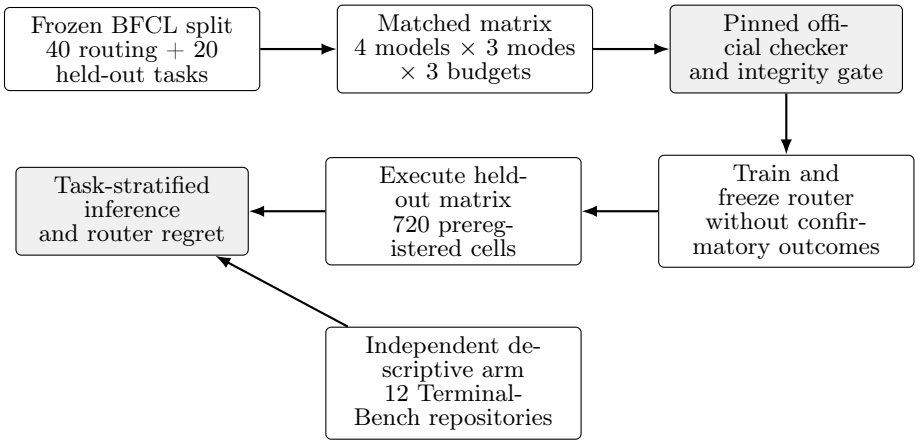


Figure 1 Frozen evidence flow. Routing tasks train the policy; confirmatory tasks estimate the budget–granularity response surface; validation tasks test only the central budget; Terminal-Bench is a separate descriptive repository-level arm. Arrows show information flow, not causal dependence. No confirmatory or validation label enters router training.

The BFCL subset contains four official categories: simple calls, multiple calls, parallel calls, and long-context calls. Scoring uses `bfcl-eval==2026.3.23` from Gorilla revision `6ea57973c7a6` (the full hash is in the artifact). The routing set has ten tasks per category; confirmatory analysis uses five disjoint tasks per category; the validation extension uses eight further tasks per category. The confirmatory matrix therefore contains $20 \times 4 \times 3 \times 3 = 720$ episodes. Validation contains $32 \times 4 \times 3 = 384$ episodes at $T = 1024$.

4.2 Models and orchestration modes

The frozen OpenRouter panel is `z-ai/glm-4.7-flash`, `xiaomi/mimo-v2.5`, `qwen/qwen3.7-flash`, and `nvidia/nemotron-3-nano-30b-a3b`. Provider metadata and returned model IDs are stored with the traces; model names describe endpoints in the provider snapshot retrieved on 2026-07-28, not immutable weights.

G0 receives the task and tools in one call. G1 spends the same T across a planner, executor, and reviewer; later roles receive earlier artifacts but no extra completion allowance. G3 uses a coordinator, separately prompted workers, and a synthesizer under the same ledger. All modes use the same normalized function schema, request seed 11, temperature 0, evaluator, and retry contract. The ledger checks every phase as well as the whole episode. A request-side one-token headroom handles provider accounting while the registered ledger still enforces the original allowance.

4.3 Router and unavailable actions

Candidate policies were selected by grouped cross-validation on routing tasks: a static action, a histogram gradient booster, nearest neighbours, and a linear spline model. The frozen policy is serialized with its feature schema, training hashes, and

Table 1 Confirmatory response surface. Accuracy is official BFCL correctness; invalid is the malformed-call fraction; tokens and cost include every call in an episode. Every row shares the displayed total budget T, not a per-call budget.

Mode	T	n	Accuracy	Invalid	Tokens	Cost (USD)
G0	512	80	0.037	0.963	22774.3	0.0008
G0	1024	80	0.113	0.875	43325.3	0.0016
G0	2048	80	0.150	0.850	64330.9	0.0022
G1	512	80	0.000	0.950	28087.0	0.0012
G1	1024	80	0.050	0.875	43003.2	0.0019
G1	2048	80	0.050	0.912	76757.1	0.0034
G3	512	80	0.013	0.975	28119.7	0.0014
G3	1024	80	0.000	0.963	33494.8	0.0014
G3	2048	80	0.025	0.950	51754.2	0.0023

'confirmatory_outcomes_used=false'. If all three modes are unavailable for one task–model–budget block, its ITT zeros remain in fixed-mode estimates but the block is excluded from dispatch regret because no action was available. Continuation requires at most 2% all-mode outage blocks, at most 5% missing usage, no duplicate keys, and no budget violation.

4.4 Uncertainty and multiplicity

The unit of generalization is the task, not an API call. Bootstrap resampling is stratified by the four task categories; all model and mode observations for a selected task travel together. We use 10,000 draws and the frozen seed. Percentile 95% intervals accompany effect estimates. Exact paired randomization tests are Holm-adjusted within the declared families. Model and category heatmaps are heterogeneity diagnostics, not additional confirmatory hypotheses. The validation set was designated for a separate analysis rather than pooling; because its matrix stopped incomplete, that quality analysis was not performed.

5 Confirmatory BFCL results

The audited matrix contains 720 episodes and 5622 calls. Overall official accuracy is 0.049; the provider cost is USD 1.28. There are 0 completion-budget violations, 20 transport failures, and 0 evaluator exceptions. Table 2 reports the paired estimands rather than differences between unrelated task averages.

Absolute performance is low and must not be hidden by averaging. G0 accuracy rises from 0.0375 at $T = 512$ to 0.1125 at 1024 and 0.150 at 2048. G1 yields 0, 0.050, and 0.050; G3 yields 0.0125, 0, and 0.025. Across the nine cells, 0.85–0.975 of episodes contain an invalid final function call. The same panel scored 0.351 on the disjoint routing set, so the confirmatory result also reveals severe task-transfer difficulty; routing accuracy is not pooled with held-out accuracy. The surface is therefore floor-dominated: it estimates this registered low-budget panel, but cannot identify a favourable decomposition region that may exist once the unified baseline is reliably functional.

Per budget doubling, G1 changes its paired effect by -0.031 [-0.062, -0.000] (Holm $p = 1.000$), and G3 by -0.050 [-0.094, -0.012] (Holm $p = 1.000$). These slopes answer

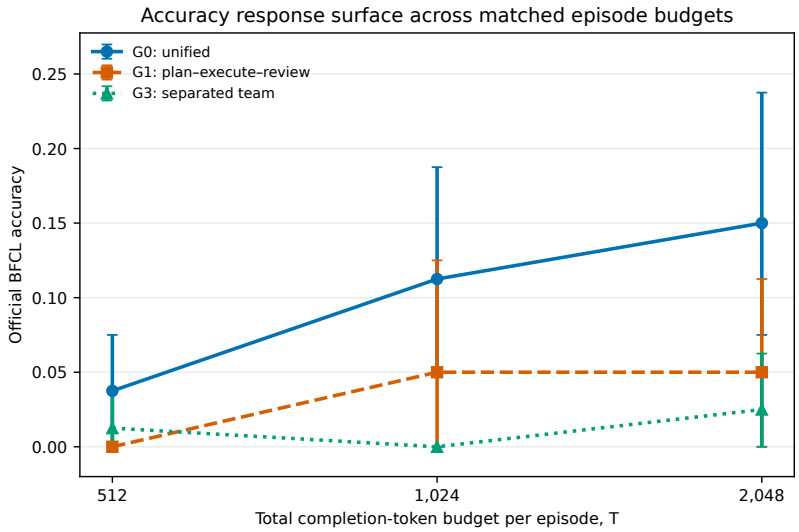


Figure 2 Official accuracy against the matched episode budget. Points are task-weighted means; bars are category-stratified task-bootstrap 95% intervals. Colour, marker, and line style redundantly encode the mode, so the figure remains interpretable in greyscale. Connecting lines describe the registered budgets only and must not be read as continuous scaling laws.

Table 2 Paired decomposition effects relative to G0. Positive values favour the decomposed mode. Intervals resample tasks within category; p-values are Holm-adjusted within the six registered mode–budget comparisons.

Mode	T	Δ accuracy	95% CI	Holm p
G1	512	-0.037	$[-0.075, 0.000]$	0.160
G1	1024	-0.062	$[-0.125, 0.000]$	0.160
G1	2048	-0.100	$[-0.163, -0.050]$	0.004
G3	512	-0.025	$[-0.050, 0.000]$	0.166
G3	1024	-0.113	$[-0.188, -0.050]$	0.004
G3	2048	-0.125	$[-0.212, -0.050]$	0.004

the interaction question: they do not imply that increasing tokens has the same effect outside the registered range.

H1 is not supported: both estimated interactions point opposite to the preregistered positive direction (one-sided adjusted $p = 1.000$). At $T = 2048$, G1 loses 0.100 $[-0.163, -0.050]$ and G3 loses 0.125 $[-0.213, -0.050]$ relative to G0. G3 is also negative at $T = 1024$ (-0.113 $[-0.188, -0.050]$); all three adjusted $p < 0.005$. No comparison satisfies the predeclared ± 0.03 equivalence criterion. These tests answer RQ1–RQ2 only within the registered floor-dominated range. RQ3 is addressed descriptively by Figures 4 and 5; it is not a family of confirmatory subgroup tests.

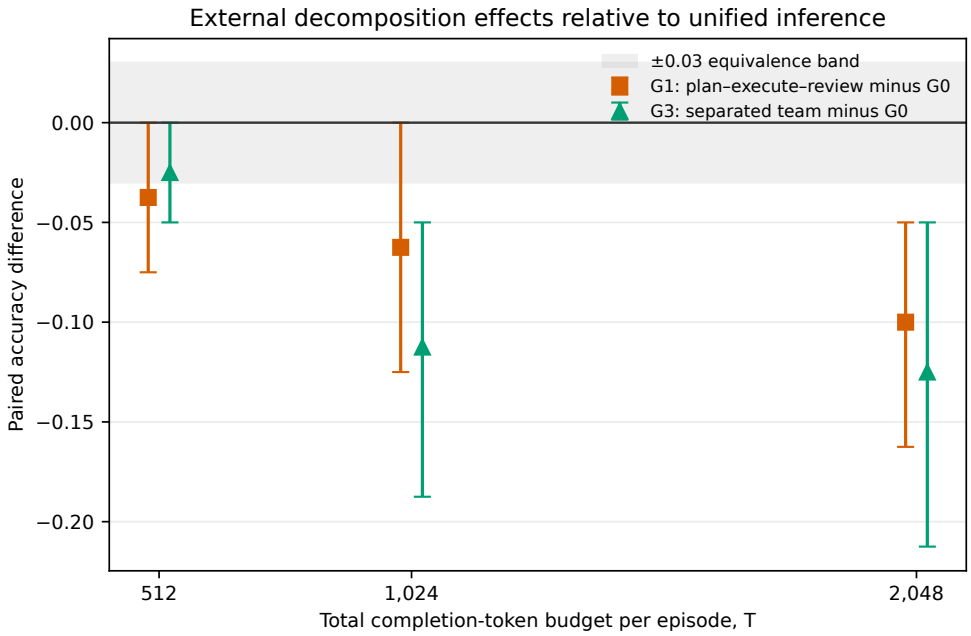


Figure 3 Paired effects from Table 2. The grey band marks the predeclared ±0.03 practical-equivalence region; crossing zero means the data do not identify a direction at the displayed uncertainty.

Table 3 Held-out routing policies on confirmatory tasks. Utility is defined by Eq. (3); regret is relative to an oracle that chooses only among already executed actions. Lower regret is better.

Policy	Accuracy	Utility	Regret	Tokens
Always G0	0.100	-0.145	0.024	43476.8
Best static	0.100	-0.145	0.024	43476.8
Frozen router	0.100	-0.145	0.024	43476.8
Oracle	0.113	-0.121	0.000	32968.3

6 Held-out routing and validation stop

The preregistered H3 contrast (frozen-router regret minus best-static regret) is 0.000 [0.000, 0.000]. Task labels are held out, but the test is degenerate because the learned and static policies never diverge. Feature extraction is deterministic and its engineering cost is not included.

In fact, the frozen router, best-static policy, and always-G0 select the same actions here: each has accuracy 0.100 and mean regret 0.024. The oracle reaches only 0.113. Thus H3 is not supported, and the small oracle gap leaves little attainable routing headroom on these tasks.

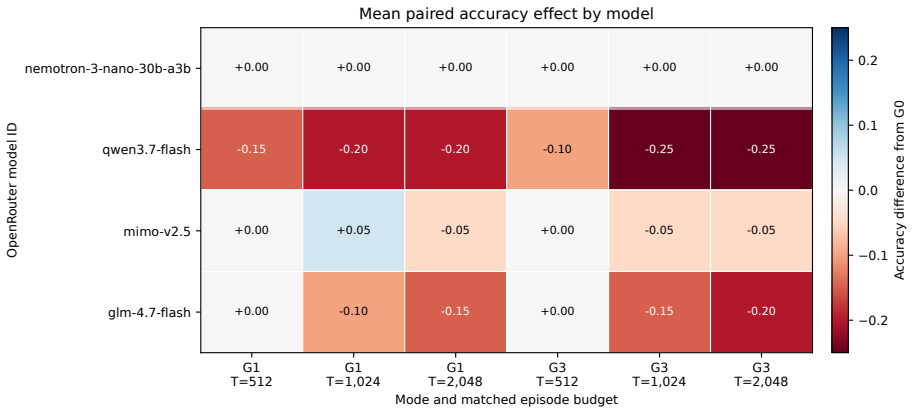


Figure 4 Mean paired effect by endpoint, mode, and budget. Each cell prints its value; the diverging scale is centred at zero. This is a heterogeneity map, not a ranking of model quality, and no cell is treated as an independent confirmatory test.

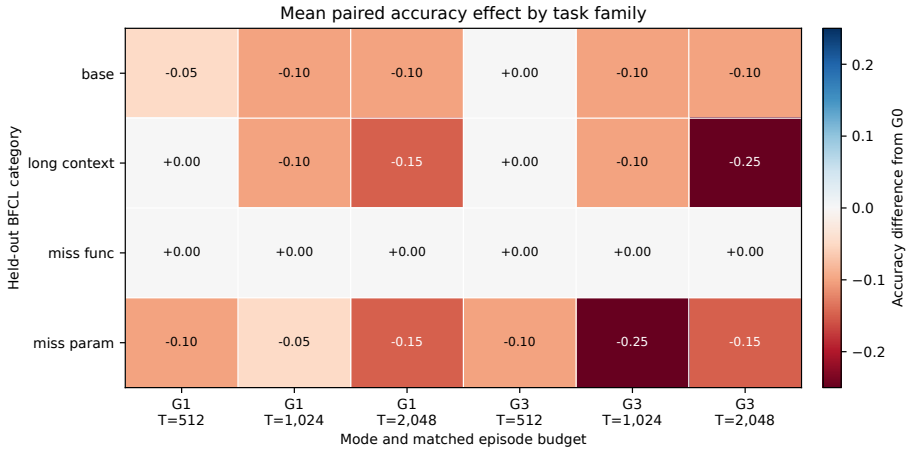


Figure 5 Paired effect by official task category. Values compare the same task–model–budget blocks with G0. Sparse category cells can be unstable, so the plot diagnoses boundary conditions rather than proving category-specific laws.

The independent validation was designed for 32 new tasks at $T = 1024$ (384 episodes). The disclosed one-time USD 3.60 hard ceiling stopped dispatch after 352 episodes; the last 32 were never generated. Following the preregistered stop rule, the ceiling was not extended again, official scoring was not run, and no partial validation aggregate is reported. These transport-only traces remain public so the incompleteness is auditable. The registered seed sensitivity check is also omitted because the same ceiling was reached. Consequently, the second-set part of RQ4 is unanswered rather than negative; these traces are evidence of protocol adherence, not replication.

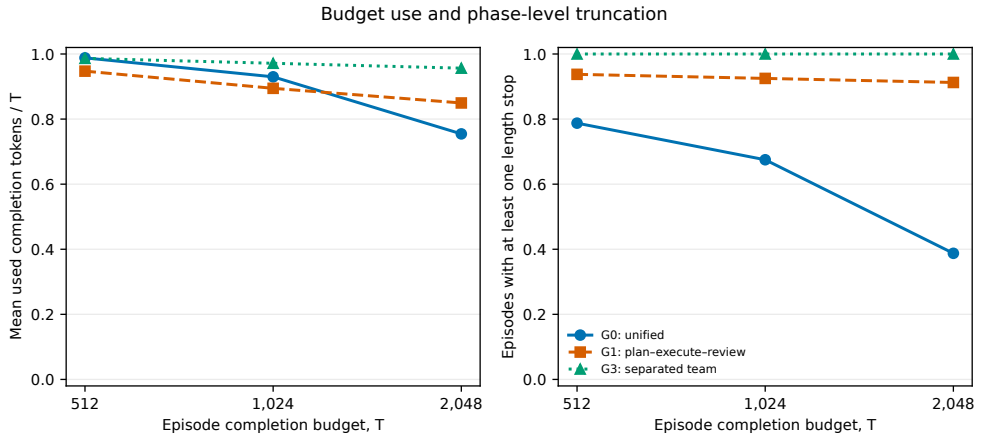


Figure 6 Budget pressure by mode and registered ceiling. Left: mean fraction of the total completion ledger consumed. Right: fraction of episodes with at least one phase ending at a length stop. Both are mechanism diagnostics, not accuracy measures.

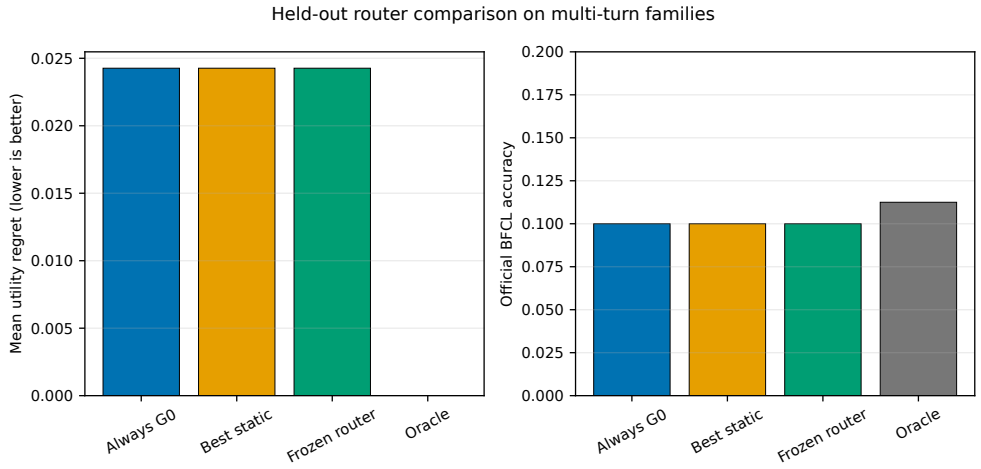


Figure 7 Accuracy and regret of the frozen policies. The oracle is an unattainable upper reference because it uses the observed outcome of every mode; it is not a deployable competitor.

7 Repository-level stress test

BFCL isolates structured tool selection but does not reproduce editing, testing, version control, and container failures. We therefore add a separate, descriptive Terminal-Bench 2.0 arm using the pinned Harbor framework and one `codex-gpt-5.6-sol` agent configuration. Twelve tasks were selected and frozen before execution. This arm is not a topology comparison: Codex does not expose a matched internal completion ledger, and the sample is too small for a leaderboard claim.

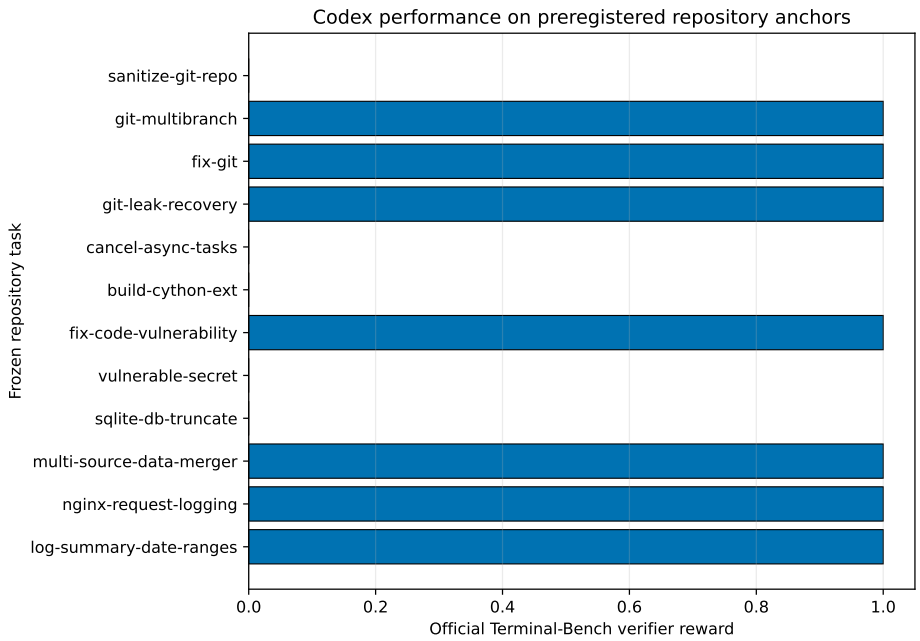


Figure 8 Outcome of every frozen Terminal-Bench task. Green denotes verifier pass; other marks distinguish agent refusal, environment failure, and verifier failure. Missing infrastructure is not silently converted into agent incorrectness, while the ITT pass rate retains all 12 tasks in its denominator.

The agent passes 7/12 tasks: rate 0.583 with Wilson 95% interval $[0.320, 0.807]$. There are 4 infrastructure failures. Raw Harbor results, verifier output, and trajectories are published; credential-shaped benchmark fixtures are deterministically redacted and their original/published hashes are recorded. The result is evidence that the artifact operates on real repositories, not evidence that the proposed granularity policy improves a repository agent.

8 Discussion

The experiment separates three questions that are often collapsed. First, whether decomposition helps at one budget is answered by $\Delta_g(T)$. Second, whether additional budget changes the answer is captured by the log-budget interaction. Third, whether those differences are predictable before execution is the held-out routing problem. A favourable oracle gap is merely opportunity; only the frozen-router contrast measures attainable selection.

Two practical rules follow without claiming universality. A decomposed workflow should report one ledger across all calls, because per-call limits silently multiply compute. It should also be compared with an always-unified and best-static baseline before adaptive routing is credited. Conversely, a negative low-budget effect does not show that collaboration is useless: it can identify a region where coordination overhead consumes the allocation.

9 Threats to validity and limitations

Construct validity. BFCL correctness captures structured function selection and arguments, not long-horizon autonomous success. G1/G3 are controlled prompt topologies, not all possible multi-agent systems. Token counts come from provider usage and can change with endpoint accounting.

Internal validity. Pairing, a shared evaluator, frozen prompts, and a hard ledger reduce confounding. They do not remove provider nondeterminism, endpoint drift, or prompt sensitivity. G1/G3 divide an already small ledger across more calls and expose more transport surfaces; their observed loss therefore combines topology, naive budget splitting, truncation, and call reliability rather than isolating an abstract "collaboration" effect. One all-mode routing outage block was excluded only from dispatch regret under a rule frozen before confirmation.

Statistical conclusion. Twenty confirmatory tasks give only five clusters per category and one request seed per cell; even 10,000 bootstrap draws cannot create task or run-to-run information absent from the design. Heatmaps and repository outcomes are descriptive. Equivalence is claimed only when the predeclared TOST condition holds.

External validity. The panel contains four hosted endpoints, selected BFCL categories, a registered 32-task central-budget validation matrix that stopped at 352/384 unscored episodes, one provider, one Terminal-Bench agent, 12 repository tasks, and one Windows/Docker host. There is no direct MaAS/BAMAS/MACA implementation comparison, independent replication, energy measurement, or official leaderboard submission. Endpoint selection prioritised protocol compliance and affordable completion of the registered matrix, not representativeness of the model market.

Reproducibility boundary. APIs are mutable. We publish manifests, prompt hashes, returned model IDs, raw calls, usage, costs, evaluator version, container/repository commits, analysis seeds, and generated figures. This supports forensic reproduction but cannot make a hosted model immutable.

10 Conclusion

This work turns the broad question "are more agents better?" into three auditable quantities: a paired effect at a fixed shared budget, its change per budget doubling, and held-out routing regret. The answer is necessarily local to the frozen, floor-dominated panel: decomposition did not pay at any registered budget, and the frozen router collapsed to the unified static policy. These are negative boundary results, not a universal MAS comparison. The accompanying repository arm demonstrates end-to-end agent execution while making its different estimand explicit. Together, the preregistration order, real tool calls, budget ledger, transparent validation hard stop, failure accounting, and bilingual artifact make the study stronger and more reusable; they do not justify a guarantee of A*-venue acceptance.

Data and code availability

The complete artifact is available at https://github.com/daniil-novel/article-AdaptiveAgentGranularity and is pinned by release tag `v2.0.0`. Pre-registration tags, raw JSONL, official scores, frozen router, statistical outputs, Terminal-Bench traces, generated tables/figures, and both TeX/PDF versions are versioned together. 'REPRODUCIBILITY.md' gives the exact command order and distinguishes confirmatory, validation, descriptive, and Study 0 evidence.

Appendix A Study 0: controlled synthetic precursor

Before the real-benchmark phase, 40 synthetic Python function-synthesis tasks were executed on five local Q4 models in four prompt regimes under a 384-completion-token ceiling (1,760 cells). It found that external multi-call pipelines frequently exhausted split budgets and motivated the budget sweep. Because Study 0 lacks real tools and its outcomes informed the new design, it is not pooled with BFCL and is not confirmatory evidence for H1–H3. Its preregistrations, raw data, analysis, and the v1.0.0 paper remain in the repository.

References

[1] Chen, M., Tworek, J., Jun, H., et al.: Evaluating Large Language Models Trained on Code. arXiv:2107.03374 (2021)

[2] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629 (2022)

[3] Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366 (2023)

[4] Madaan, A., Tandon, N., Gupta, P., et al.: Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651 (2023)

[5] Liu, Z., Zhang, Y., Li, P., Liu, Y., Yang, D.: A Dynamic LLM-Powered Agent Network for Task-Oriented Agent Collaboration. arXiv:2310.02170 (2023)

[6] Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K.: SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770 (2023)

[7] Zheng, L., Chiang, W.-L., Sheng, Y., et al.: Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv:2306.05685 (2023)

[8] Zhuge, M., Wang, W., Kirsch, L., Faccio, F., Khizbullin, D., Schmidhuber, J.: GPTSwarm: Language Agents as Optimizable Graphs. In: ICML (2024)

[9] Wang, Q., Wang, T., Li, Q., Liang, J., He, B.: MegaAgent: A Practical Framework for Autonomous Cooperation in Large-Scale LLM Agent Systems. arXiv:2408.09955 (2024)

[10] Zhuo, T.Y., Vu, M.C., Chim, J., et al.: BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. arXiv:2406.15877 (2024)

[11] OpenAI: Introducing SWE-bench Verified. https://openai.com/index/introducing-swe-bench-verified/ (2024)

[12] Gu, J., Jiang, X., Shi, Z., et al.: A Survey on LLM-as-a-Judge. arXiv:2411.15594 (2024)

[13] Aleithan, R., Xue, H., Mohajer, M.M., Nnorom, E., Uddin, G., Wang, S.: SWE-Bench+: Enhanced Coding Benchmark for LLMs. arXiv:2410.06992 (2024)

[14] Zhang, G., Yue, Y., Li, Z., et al.: Cut the Crap: An Economical Communication Pipeline for LLM-based Multi-Agent Systems. In: ICLR (2025)

[15] Hu, S., Lu, C., Clune, J.: Automated Design of Agentic Systems. In: ICLR (2025)

[16] Zhang, J., Xiang, J., Yu, Z., et al.: AFlow: Automating Agentic Workflow Generation. In: ICLR (2025)

[17] Zhang, G., Niu, L., Fang, J., Wang, K., Bai, L., Wang, X.: Multi-agent Architecture Search via Agentic Supernet. In: ICML (2025)

[18] Sun, H., Zeng, S.: Introspection of Thought Helps AI Agents. arXiv:2507.08664 (2025)

[19] Zan, D., Huang, Z., Liu, W., et al.: Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving. arXiv:2504.02605 (2025)

[20] Kim, Y., Gu, K., Park, C., et al.: Towards a Science of Scaling Agent Systems. arXiv:2512.08296 (2025)

[21] Zhou, H., Wan, X., Sun, R., et al.: Multi-Agent Design: Optimizing Agents with Better Prompts and Topologies. In: ICLR (2026)

[22] Lu, Y., Hu, Y., Zhao, X., Cao, J.: DyTopo: Dynamic Topology Routing for Multi-Agent Reasoning via Semantic Matching. arXiv:2602.06039 (2026)

[23] Yu, G.: AdaptOrch: Task-Adaptive Multi-Agent Orchestration in the Era of LLM Performance Convergence. arXiv:2602.16873 (2026)

[24] Tran, D., Kiela, D.: Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets. arXiv:2604.02460

(2026)

[25] Yang, L., Luo, J., Liu, X., Lou, Y., Chen, Z.: BAMAS: Structuring Budget-Aware Multi-Agent Systems. In: AAAI, 40(35), 29802–29810 (2026)

[26] Li, H., Chen, S., Sun, S., Wang, H.: Multi-Agent Coordination Adaptation via Structure-Guided Orchestration. arXiv:2605.25746 (2026)

[27] Hamri, M., Talgam-Cohen, I.: ZEBRA: Zero-shot Budgeted Resource Allocation for LLM Orchestration. arXiv:2605.20485 (2026)

[28] Zhang, H., Li, Y., Liu, Z., Liu, C., Zhang, S., Zhou, Y.: Adaptive Confidence Gating in Multi-Agent Collaboration for Efficient and Optimized Code Generation. arXiv:2601.21469 (2026)

[29] Peng, Y., Hou, H., Zhu, X., He, Y.T., Yu, F.R.: SEMAG: Self-Evolutionary Multi-Agent Code Generation. arXiv:2603.15707 (2026)

[30] Yan, F.: A Function Calling Perspective on Scalable Large Language Model Agent Evaluation. Technical Report UCB/EECS-2025-184, UC Berkeley (2025)

[31] Merrill, M.A., Shaw, A.G., Carlini, N., et al.: Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. In: ICLR (2026)

[32] Harbor Framework Team: Harbor: A Framework for Evaluating and Optimizing Agents and Models in Container Environments (2026). https://github.com/harbor-framework/harbor

[33] Su, H., Diao, S., Lu, X., et al.: ToolOrchestra: Elevating Intelligence via Efficient Model and Tool Orchestration. arXiv:2511.21689 (2025)

[34] Ali, M.S., Novik, A., Boddupally, A., et al.: The Harness Effect: How Orchestration Design Sets the Token Economics of Enterprise Agentic AI. arXiv:2607.06906 (2026)

[35] Khan, S.: Token Budgets: An Empirical Catalog of 63 LLM-Agent Budget-Overrun Incidents. arXiv:2606.04056 (2026)

[36] Hui, B., Yang, J., Cui, Z., et al.: Qwen2.5-Coder Technical Report. arXiv:2409.12186 (2024)

[37] Guo, D., Zhu, Q., Yang, D., et al.: DeepSeek-Coder: When the Large Language Model Meets Programming. arXiv:2401.14196 (2024)

[38] Lozhkov, A., Li, R., Allal, L.B., et al.: StarCoder 2 and The Stack v2: The Next Generation. arXiv:2402.19173 (2024)

[39] CodeGemma Team, Banks, J., Brix, C., et al.: CodeGemma: Open Code Models Based on Gemma. arXiv:2406.11409 (2024)

[40] Dubey, A., Jauhri, A., Pandey, A., et al.: The Llama 3 Herd of Models. arXiv:2407.21783 (2024)